Lian Zhu, Valente Basilio Pena

Term Project: Instant Messaging App

Architecture Document

Table of Contents

1 Introduction....2

1.1 Purpose....2

1.2 Target Audience....2

1.3 Terms and Definitions....2

2 Architectural Goals and Constraints....3

2.1 Security....3

2.2 Persistency....3

2.3 Reliability/Availability....3

3 System Overview....4

4 System Architecture....5

4.1 Front-end Architecture....5

4.2 Back-end Microservices Architecture....5

5 Database Design....6

6 Issue and Concerns....7

1 Introduction

1.1 Purpose

This document describes the system architecture of the Instant Messaging App defined in the feature document. It explains the following:

- Overall architectural design decisions
- Frontend/backend structure
- Microservices decomposition
- Database design
- Non-function requirements

Furthermore, this document serves as a blueprint for implementation, a reference for integration and a guide for future extension and maintenance.

1.2 Target Audience

- Development team members
- Professor Fei Xie and TA Huan Wu
- Future developers maintaining the system

1.3 Terms and Definitions

Client - Web application used by end users

Server - Backend system handling APIs and messaging

API - Application Programming Interface (REST endpoints)

JWT - JSON Web Token for auth

WebSocket - Persistent connection protocol for real-time messaging

Microservice - Independently deployable backend service

API Gateway - Single entry point routing requests to backend services

2 Architectural Goals and Constraints

The system will follow a layered + microservices architecture such as:
Frontend (React SPA) -> API Gateway (Express.js) -> Microservices (Node.js) -> MongoDB Database.
Real time communication is handled using WebSocket (Socket.io).

2.1 Security

The system will protect users through

- Hashing passwords
- JWT based auth
- Server-side validation
- MongoDB injection prevention

Some limitations/constraints that could exist

- Limited project timeline prevents implementation of advanced features

2.2 Persistency

The system will achieve data persistence through a MongoDB database storage containing users, conversations, messages, group membership, and mute/unread states.

2.3 Reliability/Availability

The strategies that will be applied to achieve system reliability:

- Stateless backend services
- WebSocket reconnection handling on frontend
- Database indexing for performance
- Error handling middleware in Express

Following constraints:

- Single MongoDB instance
- Internet connection required for real-time messages

3 System Overview

The Instant Messaging Application consists of:

1. User Interface Layer (Presentation Layer)

Login / Registration

Conversation List View

Chat Room View

Profile Settings

Error Feedback UI

2. User Interface Management Layer

React Router for navigation

Context API / Redux for global state

WebSocket client for real-time updates

3. Application Services Layer

Authentication Service

Conversation Service

Messaging Service

Profile Service

4. Shared Infrastructure Services

API Gateway

WebSocket Server

Logging middleware

Authentication middleware

5. Database Management Layer

MongoDB

Indexed collections for efficient retrieval

Layered Architectural Model

Presentation Layer

↓

Application Layer

↓

Service Layer (Microservices)

↓

Data Access Layer

↓

Database

This separation improves maintainability and testability.

4 System Architecture

4.1 Front-end Architecture

Frameworks and tools involve:

- React.js
- React Router
- Axios for API calls
- Socket.io-client for real-time messaging
- Git + GitHub for version control

The frontend structure will also involve /components for auth, chat, conversations, profile, /pages for login, register, home, chatroom, /services for api.js, socket.js, /context for authcontext, chatcontext. It's responsible for authentication handling, displaying conversations, real-time message rendering, unread indicators, muting conversations, error feedback display, and route protection.

Frontend Architecture Diagram

User

↓

React UI

↓

Axios → API Gateway

Socket.io → WebSocket Server

4.2 Back-end Microservices Architecture

The backend will use Node.js + Express + MongoDB (MERN stack)

Microservices breakdown:

1) API Gateway
    a) Central entry point
    b) JWT validation
    c) Request routing
    d) Rate limiting
    e) Logging
2) Auth service
    a) Registration
    b) Login
    c) Logout
    d) JWT generation
    e) Password hashing
    f) Collection of users
3) Conversation service
    a) Creating conversation
    b) Deleting conversation
    c) Add/remove members
    d) Leave group
    e) Mute/unmute
    f) Unread state tracking
    g) Database collections for conversations and conversation participants
4) Messaging service
    a) Sending messages/retrieving messages
    b) Store timestamp + sender
    c) WebSocket broadcasting
    d) Database collections for messages

5) Service Communcation
    a) REST over HTTP between client and API Gateway
    b) Internal service calls via Express routing modules
    c) WebSocket used for real-time messaging

Since all services share MongoDB:

- Each service uses its own schema model
- Data consistency maintained through service-level validation
- Changes affecting multiple collections use atomic MongoDB operations where possible

5 Database Design

The database of choice is MongoDB. The advantages it gives include the ability to create a flexible schema for conversations and messages, easy JSON mapping, it's a good fit for MERN stack, it is scalable horizontally, and it allows for a fast development cycle.

It will consist of:

Users:

```
{
  _id,
  email,
  passwordHash,
  firstName,
  lastName,
  createdAt
}
```

Conversations:

```
{
  _id,
  type: "dm" | "group",
  name,
  creatorId,
  participants: [userId],
  mutedUsers: [userId],
  createdAt
}
```

```
Messages:
{
  _id,
  conversationId,
  senderId,
  content,
  timestamp
}
```

There's a lot of interoperability between the UI and database, for example, the API responses are formatted as JSON, the frontend state mirrors the database models, real-time updates modify UI state immediately, and finally the fact that the backend ensures access control returning data.

Issue and Concerns

1) There might be some difficulty with real-time messaging due to its complexity. The WebSocket state implementation could be difficult, handling disconnects, or avoiding duplicate messages could pose a challenge.
2) Data consistency might be difficult, maybe there could be duplicate DMs, properly removing access when a user leaves a group, or even unread status accuracy.
3) There are obviously scalability limits with a single MongoDB instance, and WebSocket probably doesn't scale well.
4) Security risks such as injection attacks, or brute force login attempts, or even session hijacking could happen.